



Triển khai Chatbot lên Production

Từ POC đến hệ thống thực tế: Trade-off, Guardrails, Đóng gói

Đỗ Hoàng Hà

Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội (VNU-UET)

June 22, 2026



Nội dung chính

- 1 Bối cảnh
- 2 Kiến trúc RAG tổng quan
- 3 Model Serving
- 4 HNSW & Vector Database
- 5 Guardrails
- 6 Quy trình đóng gói Docker
- 7 Tổng kết

Từ POC đến Production

Thách thức về Quy mô

- **Chi phí:** Hàng trăm, hàng nghìn người sử dụng đồng thời. Chi phí tăng theo số lượng request.
- **Latency:** Người dùng không muốn chờ đợi.

Thách thức về Độ tin cậy

- **Hallucination:** Sai sót ở POC/demo là lỗi nhỏ, sai sót ở production là vấn đề lớn.
- **Security:** Prompt injection, PII leakage là rủi ro hiện hữu.

Nguyên lý cốt lõi

Sự ổn định và chi phí vận hành quan trọng hơn tính năng hào nhoáng của POC.

Tam giác Trade-off

Mọi quyết định kỹ thuật đều xoay quanh 3 trục chính:

Latency

Ưu tiên phản hồi nhanh / thời gian thực.

- **Đổi lại:** Chấp nhận giảm độ chính xác.
- **Chi phí:** Tốn tiền đầu tư phần cứng xịn để ép độ trễ xuống thấp.

Cost

Ưu tiên tiết kiệm ngân sách / tài nguyên phần cứng.

- **Đổi lại:** Phải chấp nhận độ trễ cao do cấu hình thấp hoặc API tier thấp.
- **Đổi lại:** Chỉ dùng được các model nhỏ, giới hạn về mặt năng lực/trí tuệ.

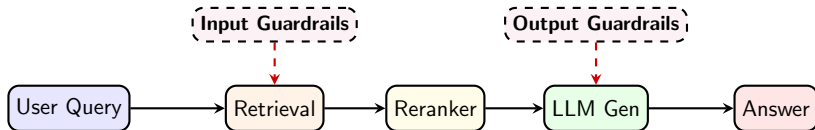
Accuracy

Ưu tiên câu trả lời đúng, sâu và chính xác nhất.

- **Gánh nặng:** Mô hình lớn, luồng xử lý phức tạp làm tăng mạnh độ trễ.
- **Chi phí:** Tiêu tốn cực kỳ nhiều VRAM và chi phí vận hành máy chủ liên tục.

Luật chơi: Chỉ chọn được 1-2 trong 3 mục tiêu đồng thời.

Kiến trúc RAG Tổng Quan



- **Input Guardrails:** Kiểm duyệt truy vấn đầu vào, ngăn chặn Prompt Injection và các nội dung độc hại trước khi chuyển tới tầng Retrieval.
- **Retrieval & Reranker:** Tìm kiếm và lọc các context chunk phù hợp nhất từ Vector DB để làm đầu vào cho mô hình.
- **Output Guardrails:** Giám sát và kiểm định trực tiếp quá trình sinh token của LLM nhằm chống hiện tượng ảo giác, rò rỉ thông tin nhạy cảm trước khi đưa ra câu trả lời cuối cùng.

Model Serving

1. Hệ thống Production

Phục vụ hệ thống lớn, lượng user cao, cần tối ưu phần cứng tối đa.

- **vLLM**

Thuật toán PagedAttention tối ưu KV Cache, đạt thông lượng cực lớn.

- **TensorRT-LLM**

Tối ưu sâu vào phần cứng NVIDIA, ép độ trễ xuống mức thấp nhất.

2. Dev / Test / R&D

Thử nghiệm nhanh, làm prototype hoặc phục vụ nghiên cứu nội bộ đơn máy.

- **Ollama**

Quản lý và deploy model gọn gàng qua CLI giống như Docker.

- **LM Studio**

Giao diện GUI trực quan, chạy local API server chỉ với 1-click.

3. Edge / Low-end

Chạy trên thiết bị ngoại vi, laptop, server không GPU hoặc tài nguyên hạn chế.

- **llama.cpp**

Viết thuần bằng C/C++, chạy mượt trên CPU/ARM với mức RAM tối thiểu.

- **ONNX Runtime**

Tận dụng đa phần cứng (NPU/DirectML), chạy tốt trên Windows AI PC.

Tư duy cốt lõi: Đừng cố chọn công cụ mạnh nhất, hãy chọn công cụ **phù hợp nhất** với hạ tầng và mục đích bài toán.

Thách thức: Tìm kiếm lân cận ở quy mô lớn

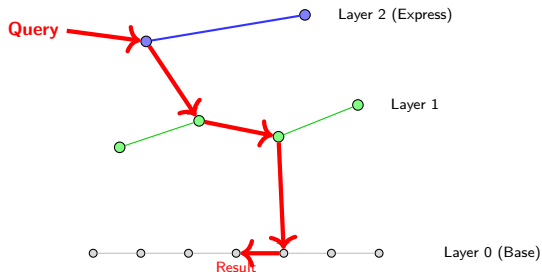
Vấn đề: Brute-force ($O(N)$)

- So sánh vector truy vấn với **tất cả** vector trong DB.
- Quy mô 1 tỉ vector → Phản hồi tính bằng giây.
- **Thảm họa** cho hệ thống Real-time.

Giải pháp: HNSW ($O(\log N)$)

- **ANN** (Approximate Nearest Neighbor).
- Cấu trúc đồ thị đa tầng.
- Tìm kiếm cực nhanh với độ chính xác cao (>95%).

HNSW (Multi-layer Graph)



Tìm kiếm từ tầng cao (nhảy xa) xuống tầng thấp (nhảy gần) để tối ưu tốc độ.

Lựa chọn Vector Database cho Production

Database	Managed	Speed	Self-host	Ưu tiên dùng khi nào?
Pinecone	Cloud	★★★★★	×	Cần zero-ops, triển khai nhanh.
Milvus	Self/Cloud	★★★★★	✓	Dữ liệu cực lớn, scale enterprise.
Qdrant	Self/Cloud	★★★★★	✓	RAG production, dễ vận hành.
Weaviate	Self/Cloud	★★★★·	✓	Prototype đến mid-scale.
Redis	Self/Cloud	★★★★·	✓	Đã có sẵn Redis infrastructure.
pgvector	Self-host	★★★★·	✓	Muốn tận dụng PostgreSQL.
ChromaDB	Self-host	★★★·	✓	Dev, local, demo.

Khuyến nghị

Sử dụng **Qdrant** cho đa số hệ RAG production, **Milvus** khi cần mở rộng quy mô rất lớn và **Pinecone** khi ưu tiên vận hành dạng dịch vụ quản lý hoàn toàn.

Benchmark Vector Databases trên Hệ thống HPC

Database	Tập dữ liệu	Thời gian Index (s)	Độ trễ query (ms)	Thông lượng (QPS)
Milvus	GIST (1M, 960-dim)	84.50	23.85	1,470
	dbpedia-openai (1M, 1536-dim)	205.41	46.87	799
	Yandex-T2I (10M, 200-dim)	110.00	2.71	11,812
	Pes2o-VE (10M, 2560-dim)	691.00	72.23	442
Qdrant	GIST	215.22	58.31	1,373
	dbpedia-openai	85.12	3.24	1,738
	Yandex-T2I	131.00	6.74	4,754
	Pes2o-VE	845.00	42.56	751
Weaviate	GIST	N/A*	30.72	2,481
	dbpedia-openai	N/A*	4.87	1,518
	Yandex-T2I	5,951.00	3.00	10,671
	Pes2o-VE	19,457.00	4.07	7,853

Nhận xét Tốc độ

Milvus lập chỉ mục nhanh nhất, đặc biệt với dữ liệu vector nhiều chiều (Pes2o-VE). **Weaviate** đạt QPS xuất sắc nhưng thất cổ chai ở khâu lập chỉ mục tuần tự đối với dữ liệu lớn.

* Nguồn benchmark: Ockerman et al., "When More Cores Hurts: The Vector Database Scaling Paradox in HPC" (2024).

Benchmark Vector Databases trên Hệ thống HPC (Tiếp)

Database	GIST (1M)	dbpedia(1M)	Yandex-T2I (10M)	Pes2o-VE (10M)
Milvus	0.961	0.996	0.876	0.982
Qdrant	0.980	0.979	0.837	0.970
Weaviate	0.988	0.986	0.738	0.918

Nhận xét Chất lượng

Ở quy mô nhỏ, cả 3 DB đều duy trì độ chính xác cao. Tuy nhiên với dữ liệu lớn, **Milvus** và **Qdrant** vượt trội về Recall so với **Weaviate**. Nguyên nhân do hai Vector DB này chia nhỏ dữ liệu thành các phân đoạn (segments) và tìm kiếm HNSW song song, thay vì dùng một chỉ mục duy nhất.

* Nguồn benchmark: Ockerman et al., "When More Cores Hurts: The Vector Database Scaling Paradox in HPC" (2024).

So sánh Triển khai & Yêu cầu Hạ tầng

Tiêu chí	Pinecone	Milvus	Qdrant	Weaviate	Redis	pgvector	Chroma
Docker Setup	–	TB	Dễ	Dễ	Dễ	Dễ	Rất dễ
Container tối thiểu	–	3	1	1	1	1	1
Thành phần phụ thuộc	Cloud	etcd+MinIO	Không	Không	Không	PostgreSQL	Không
RAM Dev	–	4–8G	1–2G	2–4G	2–4G	2–4G	<1G
RAM Prod	–	16G+	8G+	8G+	8G+	8G+	Không khuyến nghị
Scale-out	*****	*****	*****	***	***	**	*
Ops Complexity	Thấp	Cao	Thấp	TB	TB	Thấp	Thấp

Nhận xét Triển khai

Qdrant mang lại tỷ lệ hiệu năng trên chi phí vận hành tốt nhất cho đa số hệ RAG production. **Milvus** phù hợp với hệ thống quy mô cực lớn cần khả năng mở rộng mạnh. **Pinecone** giúp loại bỏ hoàn toàn gánh nặng vận hành nhưng chi phí cao hơn. **pgvector** và **Redis** phù hợp khi doanh nghiệp đã sở hữu sẵn hạ tầng tương ứng.

Input Guardrails: Lớp bảo vệ Đầu vào

Mục tiêu: Lọc, làm sạch và định hướng yêu cầu (Prompt) trước khi gửi tới LLM.

1. An ninh & Bảo mật

- **Prompt Injection:** Chặn mã độc thao túng chỉ thị hệ thống.
- **Prompt Leakage:** Ngăn chặn hành vi lừa AI tiết lộ System Prompt.

2. An toàn Nội dung

- **Moderation:** Lọc từ ngữ khiếm nhã, thô tục, quấy rối.
- **Harmful Content:** Chặn hướng dẫn gây hại hoặc vi phạm pháp luật.

3. Kiểm soát Chủ đề

- **Off-topic Filter:** Từ chối câu hỏi ngoài phạm vi nghiệp vụ.
- **Sensitive Topics:** Chặn thảo luận về chính trị, tôn giáo, tranh cãi.

Output Guardrails: Lớp kiểm soát Đầu ra

Mục tiêu: Đảm bảo phản hồi không bị ảo giác, đúng định dạng và an toàn với người dùng.

1. Chống Ảo giác (Grounding)

- **Strict Context:** Chỉ trả lời dựa trên tài liệu được cung cấp.
- **Citation Verification:** Chặn, loại bỏ các trích dẫn luật không tồn tại thực tế.

3. Kiểm soát Định dạng

- **Logits Processors:** Can thiệp tầng decoding để kiểm soát việc tạo token.
- **Structured Output:** Ép cấu trúc đầu ra (JSON/Pydantic).

2. Kiểm soát Ngôn ngữ

- **Script Filtering:** Chặn và lọc các phản hồi chứa ngôn ngữ ngoài hệ Latin.
- **Tone Alignment:** Đảm bảo văn phong chuẩn mực.

4. Hậu kiểm

- **Self-Correction:** Dùng LLM phụ đánh giá tính chính xác trước khi hiển thị.
- **PII Leak Prevention:** Chặn lộ lọt dữ liệu nhạy cảm trong câu trả lời.

"Works on my machine!" — Docker giải quyết thế nào?

Không có Docker

- **Dev:** Python 3.8 + CUDA 11.8
- **Prod:** Python 3.11 + CUDA 12.2
- Thư viện bị xung đột phiên bản chéo
- Setup tốn thời gian khi chuyển máy

Với Docker

- Môi trường được **đóng gói** cố định
- Chạy nhất quán trên **mọi máy**
- Chia sẻ dễ dàng qua image registry
- Deploy chỉ cần **1 lệnh** duy nhất

Phép so sánh

Container tàu biển — hàng hóa (*ứng dụng*) đóng vào thùng chuẩn, tàu nào (*máy chủ nào*) cũng chở được mà không cần sắp xếp lại.

3 Khái niệm Cốt lõi của Docker

Dockerfile

```
FROM python:3.11  
RUN pip install ...
```

Tập lệnh hướng dẫn
cách tạo Image

“Công thức nấu ăn”

Image

```
python-app:v1.0  
[layer 1/5 ...]
```

Bản thiết kế đóng gói
môi trường hoàn chỉnh

“Khuôn bánh”

Container

```
STATUS: running  
PORT: 8000
```

Phiên bản Image
đang chạy thực tế

“Cái bánh thật”



Quy trình Thực tế — 3 Bước để Chạy Dự án

1 Viết Dockerfile

Khai báo OS, Python, thư viện và code của dự án

2 Build Image

Đóng gói thành bản thiết kế có thể tái sử dụng & chia sẻ

3 Run Container

Khởi chạy ứng dụng với GPU, mở cổng để truy cập từ ngoài

```
# Get source code
git clone https://github.com/.../project
cd project

# Buoc 2: Build image
docker build -t project-name:v1.0 .

# Buoc 3: Chay container voi GPU
docker run --gpus all \
    -p 8000:8000 project-name:v1.0
```

Lưu ý: `--gpus all` yêu cầu cài NVIDIA Container Toolkit trên máy host.

Tổng kết: Lộ trình lên Production

5 Bước then chốt

- ❶ **Mindset Trade-off:** Không thể đạt được cả 3 tiêu chí cùng lúc.
- ❷ **Lựa chọn LLM Toolkit:** Không có công cụ tốt nhất, chỉ có công cụ phù hợp nhất.
- ❸ **Lựa chọn Vector DB:** Vector DB mạnh là xương sống của hệ thống RAG lớn.
- ❹ **Xây dựng Guardrails:** Kiểm duyệt chặt chẽ Input/Output.
- ❺ **Đóng gói Docker:** Đảm bảo tính nhất quán "Build once, run anywhere".



Xin trân trọng cảm ơn!

Thảo luận và Trả lời câu hỏi

Triển khai Chatbot lên Production

Q&A Session Open

